

Inigo Quilez :: articles :: improved texture interpolation - 2009

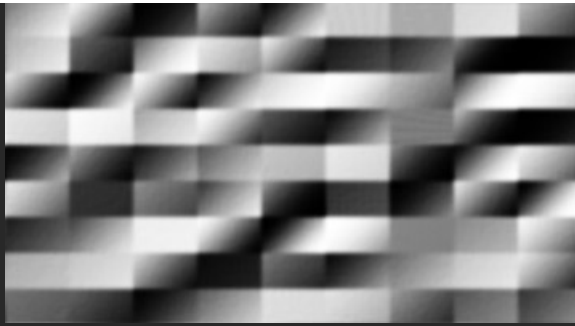


Intro

Current graphics cards are very powerful, but only because they do "simple" things. In other words, GPUs only have fast implementations for those things that are expected to make a game look great. No complain about that of course. But when something else is needed, and that something else is a little bit out of the usual requirements for a game, things do not go that well anymore. For example, texture filtering is one of those things that look great in games, but in reality the quality of current implementations is quite poor for other purposes. This article is about how to improve the linear filtering implemented in hardware.

The Thing

Linear interpolation is of course much better than the simpler nearest-rounding way of sampling a texture. However, the linear interpolation is only a first order polynomial interpolation, and therefore the slope/derivative of the resampled texture is piecewise constant. That means that when using a linearly interpolated heightmap texture to generate some normal/bump map or emboss filter one gets discontinuities (cause a normal/bump map or an emboss filter are differential operators). The effect is visible on the image below, where the left image clearly shows a cell pattern which matches the underlying texel arrangement of the original texture (see in the bottom of the articles for the texture to which the emboss was applied). The right image shows the result of applying the technique described here. Note the artifacts happen when the texture is magnified.



Regular hardware texturing, plus emboss/bump-map filter



The improved texturing technique, plus emboss/bump-map filter

Usually one should use bicubic interpolation (or even higher order interpolators) to get smooth results. But that requires extra hardware, and more importantly, a lot more bandwidth, because not only four texels must be accessed each time as in linear interpolation, but sixteen. Today GPUs are pretty much limited by the memory access times, so multiplying by four the amount of bandwidth required for one texture fetch doesn't sound like a good idea (you can have a look to cubic interpolation in [wikipedia article](#)). So, what we are doing to do is to live with the bilinear filtering provided by the GPUs:

```
vec4 getTexel( sampler2D sam, vec2 p )
{
    return texture( sam, p );
}
```

There is however a trick that is quite known to shader writers, and that's the one of using smoothstep() as a intermediate step when doing linear interpolations(not in the context of texels this time). Smoothstep is usually applied to fade the interpolation parameter before feeding mix() or lerp(). And because it's a smooth varying curve, it removes the sharp edges of the linear interpolation. Indeed, derivatives of smoothstep-ed functions are fixed (zero in this case) at the interpolation boundaries, since

$$\text{sm}(x) = x^2(3-2x)$$

and therefore

$$sm'(x) = 6x(1-x)$$

which gets value 0 both for $x=0$ and $x=1$, which are the extrema of the usual interpolation range (0..1).

Therefore, we could potentially apply the same smoothstep() technique to texture interpolation if we had a way to trick the hardware about the interpolation factor, which is of course computed deep in the hardware's texture units. We can of course not access this part of the hardware (not yet at least), but we can artificially modify the texture coordinates passed to texture2D() to get the same effect. Let's see how.

When the hardware receives a texture coordinate p thru the *texture2D()* function, it first computes the texel in which the sampling point falls. That will most often be *in-between* texels, and that's where the interpolation comes into the game. The four closest texels are chosen based on the integer part i of p , and based in the proximity to each of the texels, the four colors are blended to give a final color. The proximity is directly extracted from the fractional part f of the sampling position ($p = i + f$), and it's that direct relationship what the linearity represents, and it's that what we want to cheat. We would like the influence of the four texels to smoothly fade out as the sampling point is further, but not in a linear manner. So we have to pre-distort the fractional part with a fade curve, and that's where the smoothstep can help. Other curves than smoothstep can be used of course. I prefer to use a quintic degree curve which not only has null derivatives on the interpolation extrema, but also has second order derivatives that get zero there. That ensures than the lighting (which depends on normals) looks smooth all the time.

$$sm(x) = x^3(6x^2 - 15x + 10)$$

The code to implement the trick must extract the integer and fractional parts of the texture coordinate, tweak the fractional part with a fade curve, and assemble again the integer and fractional parts to recompose the new texture coordinate. It looks like this:

```
vec4 getTexel( sampler2D sam, vec2 p )
{
    float texResolution = float(textureSize(sam,0).x);
    p = p*texResolution + 0.5;
    vec2 i = floor(p);
    vec2 f = p - i;
    p = i + f*f*f*(f*(f*6.0-15.0)+10.0);
    p = (p - 0.5)/texResolution;
    return texture( sam, p );
}
```

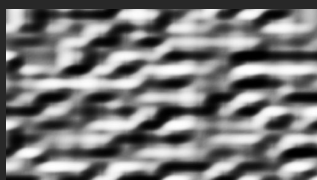
It is of course much more expensive than a regular *texture2D()* call, but there are situations where there is no

other solution to the problem, and in the end it only makes one texture fetch. As GPUs become more and more faster on computations but slower and slower on memory accesses, this code will not be that much of a problem for future GPU families.

The differences in texture quality and lighting are enormous. First row shows the described technique in action, while bottom row shows the regular GPU texture interpolation:



*Quintic fade curve
applied to hardware
interpolation*



*Emboss/bump-map filter
applied to left texture*



*Regular linear
interpolation of current
hardware*



*Emboss/bump-map filter
applied to left texture*

A live full implementation of the technique can be found here: <https://iquilezles.org/articles/texture/>

